

App Development Project Report

App Dev Project Report

1. Student Details

Name: Prajjwal Singh

Roll Number: BS24F2003522

Email: prajjwal.singh@onlinedegree.iitm.ac.in

About Me: I am a student and a full-stack development enthusiast currently working on the Trekking Management Application V2. My interests lie in backend jobs, Celery + Redis scheduling, JWT authentication, and Vue.js frontend integration.

2. Project Details

Project Title: Trekking Management Application V2

Problem Statement:

To design and build a web-based trekking management system that allows admins, trek staff, and trekkers to interact through dashboards, manage treks, bookings, and generate automated reports.

Approach:

The app was built using Flask as the backend framework with Vue.js as the frontend. It integrates JWT-based authentication, role-based access control, Redis caching, and Celery background jobs for reminders and reports.

3. AI/LLM Declaration

I used Microsoft Copilota and VS Code Chat to assist in:

- Writing SQLAlchemy model definitions
- Debugging Celery worker and Redis integration
- Drafting API documentation and report formatting

Extent of AI/LLM usage: Limited to code suggestions, debugging help, and documentation structuring. All final implementation, testing, and integration were done manually

Backend -30%, **Frontend** -40%, **Styling/UI** -8% ,**Auth + CRUD Logic** -15%

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	ORM for SQLite database
Flask-JWT-Extended	JWT authentication and role-based access
Celery + Redis	Background jobs, reminders, CSV export
Flask-Caching	API performance optimization
WeasyPrint	PDF report generation
Vue.js	Frontend framework for dashboards
Bootstrap	Styling and responsive UI
Flask-CORS	Cross-origin resource sharing

5. Database Schema / ER Diagram

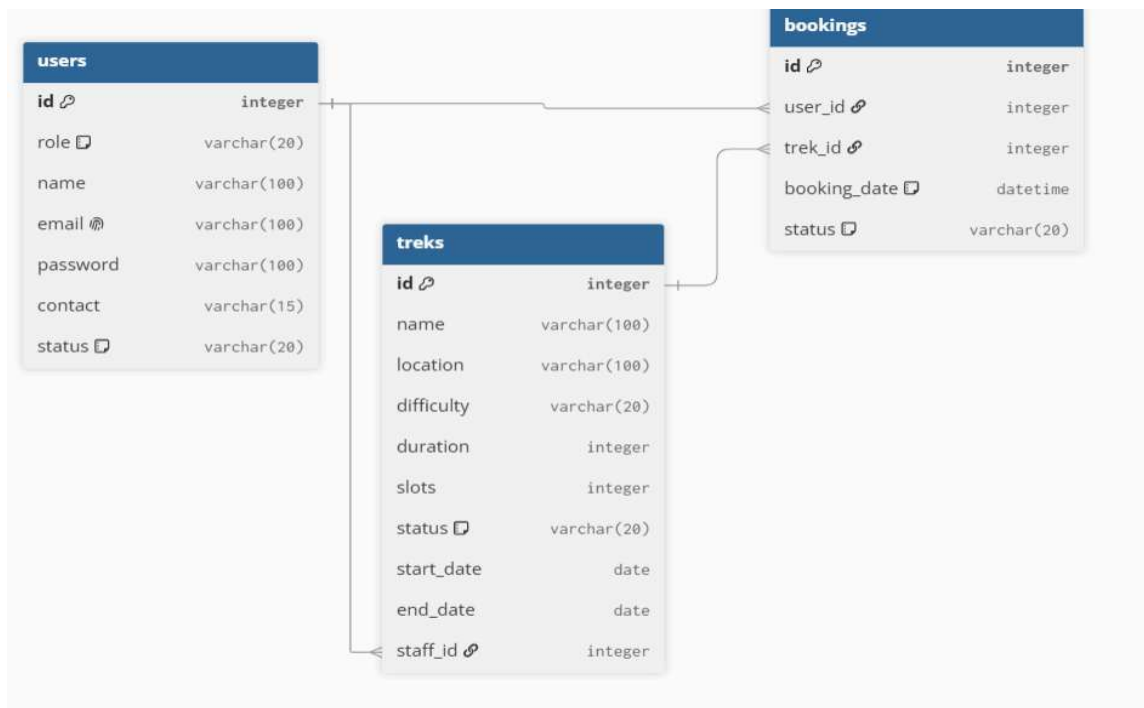
Tables:

- **User** — id, name, email, password, role (Admin / Trek Staff / Trekker)
- **Trek** — id, trek_name, difficulty, duration, available_slots, assigned_staff, status
- **Booking** — id, user_id, trek_id, booking_status, booking_date, payment_status
- **StaffProfile** — id, staff_name, trek_id, assigned_treks

Relationships:

- One-to-Many → User → Booking
- One-to-Many → Trek → Booking
- One-to-Many → Trek Staff → Trek

Reasoning: The schema ensures clear separation of roles, prevents duplicate bookings, and supports trek status tracking.



6. API Resource Endpoints

Authentication

POST /api/login

- Authenticate user and generate JWT token.

POST /api/register

- Register a new user (Admin, Staff, or Trekker).

Trek Management

GET /api/treks

- Fetch list of available treks.

POST /api/trek/add

- Admin adds a new trek.

PUT /api/trek/{id}/update

- Update trek details (slots, status, staff assignment).

DELETE /api/trek/{id}/delete

- Remove a trek (Admin only).

Booking System

GET /api/bookings

- Fetch bookings for the logged-in user.

POST /api/booking/create

- Create a new trek booking.

PUT /api/booking/{id}/cancel

- Cancel an existing booking.

Staff Management

GET /api/staff

- Fetch trek staff list.

POST /api/staff/assign

- Assign staff to a trek.

Reports & Exports

GET /api/reports/monthly

- Generate monthly trekking report (PDF via WeasyPrint).

POST /api/export/csv

- Trigger CSV export of trekking history (async Celery task).

Background Jobs (Celery + Redis)

GET /api/reminders/trek

- Send trek reminders to users.

GET /api/reports/schedule

- Schedule monthly report generation.

7. Architecture and Features (optional)

Architecture Overview:

- `app.py` – main Flask application entry point
- `/models` – SQLAlchemy models
- `/routes` – Flask Blueprints for user, trek, booking APIs
- `/templates` – Vue.js frontend templates
- `/static` – CSS, JS, Bootstrap, and chart files

Implemented Features:

- Role-based dashboards (Admin, Staff, Trekker)
 - Trek booking system with duplicate prevention
 - Trek status tracking (Open / Closed / Completed)
 - Automated trek reminders via Celery jobs
 - Monthly PDF reports for Admin
 - User-triggered CSV export with async notifications
 - Redis caching for frequently accessed endpoints
-

8. Video Presentation

Drive Link:

 Video Project 1.mp4
